

Lab 03: Tele-Communication System Design

Team Members

Uma Kumari	24B1026
Gurnoor Sohal	24B1042
Shreshtha Gupta	24B1033
Rishika Dhiman	24B1090

1. Design

1.1. Overview

We designed a visual tele-communication system in which binary messages are transmitted from a sender to a receiver using **four different colors**. The system has two separate interfaces: a **Sender Portal** and a **Receiver Portal**.

The sender takes a variable-length binary message as input. The message is encoded with error-detection and error-correction information and then converted into a sequence of colors. These colors form the communication channel between the sender and receiver. The receiver enters the observed colors and reconstructs the original binary message.

The design addresses the main requirements of the lab: variable-length messages, message framing, physical-layer encoding, and link-layer reliability.

Our system can be viewed as having two major communication layers:

- **Physical Layer:** Represents binary data using four visually distinguishable colors and transmits these colors sequentially.
- **Link Layer:** Handles framing of variable-length messages and provides reliability using two-dimensional parity for single-bit error detection and correction.

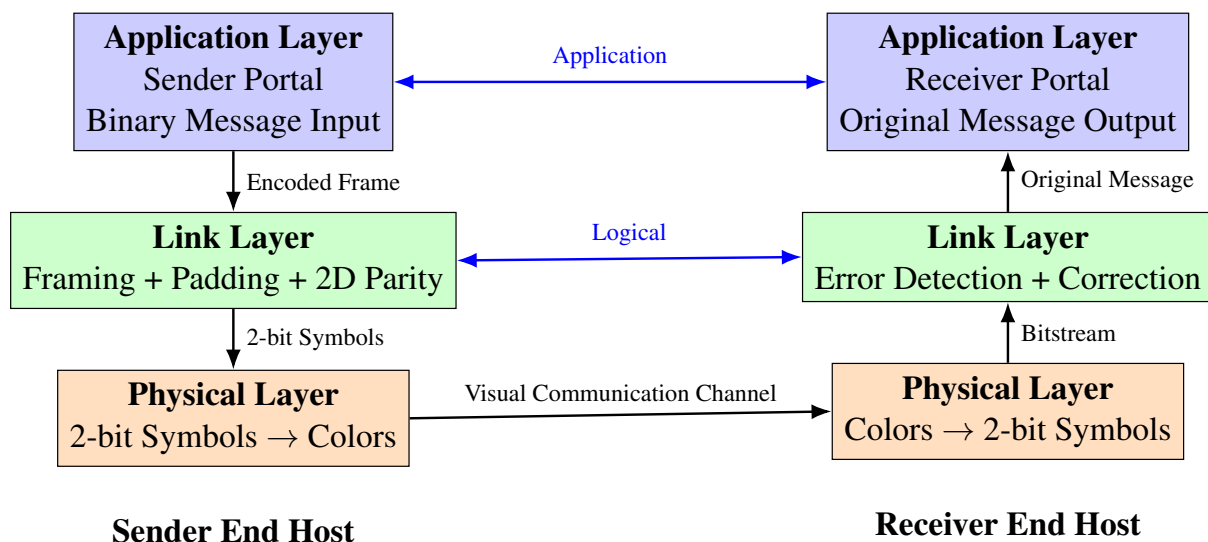
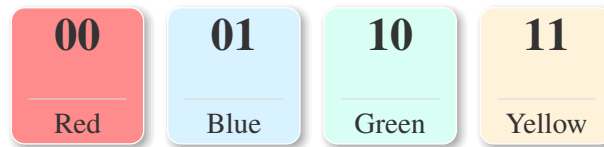


Figure 1. Layered representation of the implemented visual communication system.

1.2. Physical Layer Representation

The physical layer is represented by the transmission of **four different colors**. Each color corresponds to one of the four possible combinations of two binary bits.



Thus, every physical symbol transmitted through the visual channel carries **2 bits of information**. For example, if the encoded bitstream contains 00011110 it is divided into two-bit groups: 00 | 01 | 11 | 10 which are then represented by the corresponding four colors.

The sender displays these colors sequentially so that the physical communication channel between the sender and receiver is the sequence of observed colors. Using two bits per color reduces the number of physical symbols that have to be transmitted compared to a system that uses one color for each bit.

1.3. Link Layer Representation

The link layer is responsible for preparing the binary message for reliable transmission. In our system, the link layer performs three main functions:

1. **Framing** – identifying the beginning, end and length of a variable-length message.
2. **Error detection** – identifying whether a transmitted bit has been corrupted.
3. **Error correction** – locating and correcting a single corrupted bit.

The link-layer processing is performed before the resulting bitstream is converted into physical color symbols.

1.4. Link-Layer Framing

Since the input message can have a variable number of bits, the receiver needs a way to determine where the message ends. We solve this problem using a **6-bit length field** at the beginning of every transmission. If the original message contains an odd number of bits, an additional 0 is appended to the message. This makes the payload length even, which is required because the physical layer represents data in two-bit symbols. The padding bit is not considered part of the original message. The receiver uses the length field to remove the padding after decoding. The complete link-layer frame is:

Length | Padded Message | Row Parity | Column Parity

1.5. Link-Layer Error Detection and Correction

To provide reliability at the link layer, we use **two-dimensional even parity**. After padding, the message is placed into a rectangular grid. The program automatically selects suitable row and column dimensions for the grid. For every row, a row parity bit is calculated. Similarly, a parity bit is calculated for every column. The parity bits are calculated using XOR. For each row:

Row Parity = XOR of all bits in that row

Similarly, for each column:

Column Parity = XOR of all bits in that column

These parity bits are appended to the payload as part of the link-layer frame.

1.5.1. Error Detection

At the receiver, the received payload is reconstructed into the same grid. The receiver recalculates the row and column parity values and compares them with the parity values received in the frame. If there is no error, all calculated parity values match the received parity values. If one bit has been corrupted then exactly one row and one column would have incorrect parity. Therefore, the erroneous bit can be identified using the intersection of these two.

1.5.2. Error Correction

If the incorrect row and column are at i and j then the location of the corrupted bit is:

$$\text{Grid Index} = i \times c + j$$

where c is the number of columns. The receiver flips the bit at this location as $0 \rightarrow 1$ and $1 \rightarrow 0$. This restores the original payload. The corrected payload is then truncated using the original message length, removing any artificial padding that was added before transmission. Therefore, the receiver can both **detect and correct a single-bit error** without requiring retransmission.

1.6. Error Simulation

The sender interface contains an **error index** field. During normal transmission, this field can be left empty and no error is introduced. To testing the reliability of the system, the sender can specify the index of a bit in the original message. The encoder then flips that bit before the data is converted into colors. For example, if the original message is: 101101 and the error index is 2, the third bit is flipped before transmission. The error is therefore simulated as though it occurred on the communication medium. The receiver does not know the location of the deliberately introduced error. It uses the row and column parity checks to independently determine the location of the corrupted bit.

2. Implementation

The system was implemented using HTML, CSS and JavaScript. The JavaScript implementation contains separate encoding and decoding functions, along with the user-interface logic for the sender and receiver portals.

2.1. Sender

The Sender Portal accepts a binary message containing only 0 and 1. It performs the following operations:

1. Checks that the input contains only binary digits.
2. Stores the original message length in a 6-bit field.
3. Adds a padding zero if necessary.
4. Arranges the message into a rectangular grid.
5. Calculates row and column parity bits.
6. Combines the length, payload and parity bits into the final bitstream.
7. Optionally flips the specified error bit.
8. Groups the resulting bits into pairs.
9. Maps each pair to one of the four colors.
10. Displays the colors sequentially.

The colors are displayed with a short delay between consecutive symbols so that they can be visually transmitted to the receiver. The implementation involves displaying a colour for 1100ms and then showing a black buffer for 100ms before moving on to the next colour and repeating. The first color is displayed immediately.

2.2. Receiver

The Receiver Portal provides four buttons corresponding to the four colors. The receiver enters the colors in the same order in which they are observed. The receiver also has:

- **Backspace** – removes the last entered color.
- **Clear** – clears the complete received sequence.
- **Decode** – decodes and checks the received transmission.

During decoding, the system:

1. Converts each color back into its corresponding two bits.
2. Reads the 6-bit length field.
3. Reconstructs the expected grid.
4. Separates the payload, row parity and column parity.
5. Recalculates the row and column parity.
6. Identifies the incorrect row and column.
7. Locates the erroneous bit at their intersection.
8. Corrects the bit.
9. Removes the artificial padding.
10. Displays the corrected original message.

The receiver also displays the index of the detected erroneous bit, using **0-based indexing** with respect to the original message.

2.3. Error Handling

The implementation performs several validation checks. It rejects:

- Messages containing characters other than 0 and 1.
- Messages longer than 63 bits.
- Invalid error indices.
- Insufficient received data.
- Transmissions whose length does not match the expected encoded length.

If the parity results do not correspond to either a valid transmission or a single-bit error, the receiver reports an unsupported or invalid error pattern instead of attempting an unreliable correction.

3. Complete Communication Flow

The complete operation of our system can be summarized as follows.

3.1. Sender Side

The sender starts with the original binary message.

Original Message

The message length is encoded using 6 bits and padding is added if necessary.

Length + Padded Message

The padded message is arranged into a grid and row and column parity bits are calculated.

Length | Padded Message | Row Parity | Column Parity

This produces the complete link-layer frame. The resulting bitstream is then divided into two-bit symbols:

00 | 01 | 10 | 11 | ...

Each two-bit symbol is mapped to a color and the colors are displayed sequentially.

3.2. Communication Channel

The physical communication channel consists of the sequence of colors displayed by the sender and observed by the receiver. The laptop is used only for local computation and for displaying/entering the symbols. The actual communication between the two sides takes place through the visual color channel.

3.3. Receiver Side

The receiver enters the observed colors using the color buttons. The colors are converted back into their corresponding two-bit values, reconstructing the transmitted bitstream. The receiver then extracts:

Length | Padded Message | Row Parity | Column Parity

The parity information is checked to determine whether an error occurred. If there is a single-bit error, its row and column are identified and the corresponding bit is flipped. Finally, the receiver removes the padding and outputs the original binary message.

4. Evaluation

The system can be evaluated using messages of different lengths and by introducing errors at different positions.

4.1. Correctness

For each test message, we verify that: **Original message = Corrected message at receiver**
Tests should include:

- Messages with even length.
- Messages with odd length.
- Very short messages.
- Messages close to the maximum supported length.
- Transmission without an error.
- A single error at the beginning, middle and end of the message.

For an error-free transmission, the receiver should report: `Error detected: NO` and output the original message. When a single bit is corrupted, the receiver should identify its position and produce the original message after correction.

4.2. Error Correction Testing

To test the reliability mechanism, a message can be transmitted multiple times with the error index changed for each experiment. Our system is able to correct the error regardless of the position at which it occurs. For each case, the receiver should detect one incorrect row and one incorrect column. Their intersection should correspond to the intentionally corrupted bit. The receiver should then flip the bit and produce the original message.

4.3. Throughput

The end-to-end throughput of the system was evaluated by measuring the total time taken from the generation of the message at the sender until the corrected message was successfully received and displayed at the receiver. The lab defines end-to-end throughput as:

$$\text{Throughput} = \frac{\text{Total number of transmitted message bits}}{\text{Time last message is received} - \text{Time first message is generated}} \quad (1)$$

The throughput of our implementation is 0.73 *bps*. This measurement includes the time required to display and transmit the color sequence, enter the received colors at the receiver, decode the received bitstream, perform the parity checks, correct the possible single-bit error, and display the final message.

The total transmission time depends on the number of colors in the encoded frame and receiver's manual entry of the received colors.

4.4. Latency

Latency measures the time taken for a message to travel through the communication system from the sender to the receiver and become available as the final decoded output. For our system, end-to-end latency is measured from the instant the sender begins transmitting the first color of the encoded message until the receiver successfully decodes and displays the corrected message. The measured latency of our system on average was:

Average End-to-End Latency = 15 seconds

The latency is affected by the number of colors in the encoded transmission, length of message (longer messages generally require more physical symbols and therefore increase the transmission time) and the fact that the colours at the receiver end are manually entered.

4.5. Limitations

The current implementation is designed specifically for the lab's assumption of **at most one bit error per message**. In case of 2+ simultaneous bit errors, the receiver does not attempt to perform correction because the current parity scheme is designed to uniquely locate a single-bit error. The system also relies on the receiver correctly identifying the four colors. In an actual visual communication environment, lighting conditions, viewing distance and ambiguity between colors could affect reliability. Another limitation is that the current system uses manual color entry at the receiver, so, the speed of the overall system is also affected by how quickly the receiver can observe and enter the received colors.